

Arbeitsexposé

3D Gaussian Splatting (3DGS) Scan-to-Building Information Modeling (BIM) Pipeline

Arbeitsgrundlage für das formale Exposé an den Dozenten

Stand: 10. Mai 2026

Zweck dieses Dokuments

Enthält alle technischen Details, Repositories, Paper-Referenzen, Installations-TODOs und den Pipeline-Aufbau. Dient als Vorbereitung für das formale Kurzexposé.

Verfügbare Hardware

Graphics Processing Unit (GPU): NVIDIA RTX 4000 Ada Generation – **20 GB Video Random Access Memory (VRAM)**, 6144 CUDA Cores, 160-bit GDDR6

RAM: 64 GB

Bewertung: 20 GB VRAM sind ausreichend für alle Pipeline-Komponenten (SuGaR: 12 GB Min., Gaussian Grouping: 8–12 GB, gsplat: 8 GB+). Kein Cloud-Rechner notwendig.

1. Titel und Forschungsfrage

Arbeitstitel:

3D Gaussian Splatting vs. klassische Photogrammetrie: Geometrische Validierung, Segmentierung und Centerline-Extraktion für Leitungsinfrastruktur (Scan-to-BIM)

Zentrale Forschungsfrage (BA):

Lässt sich aus einem 3DGS-Modell über SuGaR-Mesh-Extraktion und Centerline-Algorithmen eine geometrisch verwertbare 3D-Achslinie für röhrenförmige Infrastruktur ableiten, und wie genau ist diese im Vergleich zur Real-Time Kinematic Global Navigation Satellite System (RTK-GNSS)-Referenz?

Kernbeitrag (Der Automatisierungs-Vorteil)

Eigentlich ist der große Vorteil bei diesem Ansatz, dass sich lineare Objekte nicht nur in Vektorform erzeugen lassen, sondern auch als segmentierte Meshes direkt aus der realen Welt, die von vornherein georeferenziert sind.

Das ist der **Riesen-Vorteil** an dieser Arbeit: Die Pipeline wird so zusammengeführt, dass in der Praxis nur noch ein Drohnenflug und wenige Ground Control Points (GCPs) benötigt werden. Am Ende liefert die Pipeline vollautomatisch ein segmentiertes 3D-Objekt. Da dieses über 3D Gaussian Splatting und die SuGaR-Extraktion läuft, passt sich das Mesh physikalisch korrekt an die Oberfläche an und sieht geometrisch deutlich besser (und lochfreier) aus, als wenn es rein über klassische Photogrammetrie (Multi-View Stereo, MVS-Punktwolken-Triangulierung) erstellt würde. Dadurch kann im letzten Schritt eine saubere Geoinformationssystem (GIS)-Centerline extrahiert werden.

2. Pipeline-Architektur

```
Kamera-Video --> COLMAP (Structure-from-Motion, SfM) --> 3DGS (gsplat) -->
Georef. (Helmert 7P)                                     |
                                                         |
                                                         |
Gaussian Grouping (Segm.) --> SuGaR (Mesh) --> Centerline (
DGtal)
```

Umsetzung: Die gesamte Pipeline wird durch ein zentrales Python-Orchestrierungs-Skript gesteuert. Dieses übernimmt das automatische Wechseln der Conda-Umgebungen (via subprocess), das Fehler-Handling sowie das Time-Tracking für die spätere Performance-Evaluation.

Schritte im Detail:

#	Schritt	Tool	Input	Output
1	Bilderfassung	Kamera (Video)	Campus-Testfeld	Bildsequenz (JPEG)
2	SfM	COLMAP	Bilder	Sparse Point Cloud (PC) + Kameraposen
3	3DGS Training	gsplat	Sparse PC + Kameras	3DGS-Modell (.ply)
4	Georeferenzierung	Eigenes Skript	GCP-Koord. + Modell	Georef. 3DGS
5	Segmentierung	<i>Vergleichsstudie</i>	3DGS + Annotationen	Segm. Gaussians
6	Mesh-Extraktion	SuGaR	Segm. Gaussians	3D-Mesh (.obj/.ply)
7	Centerline	DGtal / Python	Kabel-Mesh	3D-Linie (Polyline)
8	Geometrische Höhenkorrektur	Python	Centerline + Radius	Korrigierte Centerline (Z-Shift)
9	GIS-Import	ArcGIS / QGIS	Georef. Linie	Shapefile / GeoJSON
10	Evaluation	Python (numpy)	Centerline vs. RTK	Root Mean Square Error (RMSE) [cm]

3. Segmentierungsvergleich (neu)

Für die Isolation der Kabel-Gaussians werden **drei Methoden** systematisch verglichen. Dies ergibt eine zusätzliche, methodisch wertvolle Forschungsfrage.

Methode	Typ	Prinzip	Training?	Paper
(A) Farb-Filterung	Baseline	Schwellwert auf Red-Green-Blue (RGB)/Hue-Saturation-Value (HSV)-Attribute der Splats	Nein	– (eigene Impl.)
(B) FlashSplat	Optimierung	2–3 manuelle 2D-Annotationen → optimale 3D-Maske	Nein	Shen et al. (ECCV 2024)
(C) Gaussian Grouping	Training	Identity Encoding via DEVA/SAM-Masken	Ja	Ye et al. (ECCV 2024)

Bewertungskriterien:

- **Segmentierungsqualität:** Intersection over Union (IoU) / Precision / Recall der isolierten Kabel-Gaussians
- **Aufwand:** Setup-Komplexität, Annotationszeit, Dependencies
- **Auswirkung auf Centerline:** RMSE-Differenz der resultierenden Achslinie

Wissenschaftlicher Mehrwert: Der dreistufige Vergleich (manuell → trainingsfrei → trainingsbasiert) zeigt systematisch, ab welcher Komplexitätsstufe sich der Aufwand für die Kabelisolation lohnt. Gleichzeitig dient die Farbfilerung als Baseline, gegen die die ML-Methoden quantitativ bewertet werden.

Referenz-Survey: He et al. (2025): “A Survey on 3D Gaussian Splatting Applications: Segmentation, Editing, and Generation.” ArXiv: 2508.09977. Umfassendster Survey zu 3DGS-Segmentierung mit 50+ Methoden – dient als Grundlage für die Methodenauswahl und das State-of-the-Art-Kapitel.

4. Scope-Trennung PA / BA

Projektarbeit (PA) – 7 European Credit Transfer System (ECTS), 4 Wochen netto

Scope: Testfeldaufbau, Datenerfassung, Proof of Concept (Schritte 1–3).

TODO	Details	Zeit
Testfeld aufbauen	Gartenschlauch + Schnüre auf Campus	1 Tag
GCPs einmessen	8–12 GCPs mit RTK-GNSS (HS-Geräte)	1–2 Tage
TLS-Referenzscan	Laserscanner-Aufnahme des Testfelds	0.5–1 Tag
Videoaufnahme	Kamerasequenz (Nadir + Oblique)	0.5 Tage
COLMAP installieren	Pre-built Windows Binary	0.5 Tage
COLMAP Pipeline	Feature Extr. → Matching → Sparse	1 Tag
MVS-Baseline	Dense Reconstruction (Metashape/COLMAP)	1–2 Tage
gsplat installieren	Conda + pip install gsplat	1 Tag
3DGS trainieren	gsplat auf COLMAP-Output	1–2 Tage
Visueller Vergleich	Screenshots MVS vs. 3DGS	1 Tag
Rechenzeit-Benchmark	Tabelle: MVS vs. 3DGS	0.5 Tage
PA schreiben	15–25 Seiten	5–7 Tage

Forschungsfragen PA:

1. Lassen sich texturlose und feine Strukturen durch 3DGS visuell vollständiger rekonstruieren als durch MVS?
2. Wie verhalten sich die Rechenzeiten der dichten Rekonstruktion?

Ergebnis PA: Validierter Datensatz, lauffähige Pipelines, erster Vergleich.

Bachelorarbeit (BA) – 15 ECTS, 10 Wochen netto

Scope: Georeferenzierung, **Segmentierungsvergleich (3 Methoden)**, Mesh, Centerline, Evaluation (Schritte 4–9).

TODO	Details	Zeit
Georef-Skript	Helmert 7P (numpy/scipy)	2–3 Tage
(A) Farb-Filterung	HSV-Schwellwert auf Splat-Attribute	1–2 Tage
(B) FlashSplat	2–3 Annotationen → 3D-Maske	2–3 Tage
(C) Gaussian Grouping	DEVA-Masken + Identity Encoding	3–5 Tage
SuGaR installieren	Conda (Python 3.9, CUDA 11.8)	1–2 Tage
SuGaR Windows-Fix	Pfadtrenner mit pathlib fixen	0.5 Tage
SuGaR Mesh	Poisson-Rekonstruktion Kabel-Gaussians	1–2 Tage
Centerline-Algorithmus	DGtal oder Python-Reimplementierung	3–5 Tage
GIS-Import	Export Shapefile → ArcGIS/QGIS	1–2 Tage
RMSE-Evaluation	Centerline vs. RTK-Referenzlinie	2–3 Tage
Wirtschaftlichkeit	Kosten/Zeit vs. klass. Vermessung	2–3 Tage
BA schreiben	40–60 Seiten	15–20 Tage

Forschungsfragen BA:

1. Wie genau ist die extrahierte Centerline im Vergleich zur RTK-Referenz (RMSE)?
2. **Welche der drei Segmentierungsmethoden (Farb-Filter, FlashSplat, Gaussian Grouping) liefert die robusteste Isolation bei geringstem Aufwand?**
3. **Praxis-Problem:** Wie lässt sich der systematische Höhenfehler der Centerline algorithmisch korrigieren (Z-Shift), der durch die unvollständige optische Erfassung des Kabels im Graben (Halbzylinder-Geometrie) entsteht?
4. Ab welchem Objektdurchmesser scheitert die Pipeline?
5. Ist 3DGS eine wirtschaftlichere Alternative für die Bauüberwachung?

5. GitHub-Repositories

Tool	URL	Lizenz
COLMAP	https://github.com/colmap/colmap	BSD-3
gsplat	https://github.com/nerfstudio-project/gsplat	Apache-2.0
SuGaR	https://github.com/Anttwo/SuGaR	Research
Gaussian Grouping	https://github.com/lkeab/gaussian-grouping	Apache-2.0
FlashSplat	https://github.com/florinshen/FlashSplat	Research
DEVA	https://github.com/hkchengrex/Tracking-Anything-with-DEVA	–
DGtal	https://github.com/DGtal-team/DGtal	LGPL
Original 3DGS	https://github.com/graphdeco-inria/gaussian-splatting	Research

6. Paper-Referenzen

Kern-Paper (direkt in Pipeline genutzt)

1. **3D Gaussian Splatting** – Kerbl et al. (SIGGRAPH 2023)
“3D Gaussian Splatting for Real-Time Radiance Field Rendering.”
ACM Transactions on Graphics 42(4). Best Paper SIGGRAPH 2023.
2. **SuGaR** – Guédon, Lepetit (CVPR 2024)
“SuGaR: Surface-Aligned Gaussian Splatting for Efficient 3D Mesh Reconstruction.”
ArXiv: 2311.12775
3. **Gaussian Grouping** – Ye et al. (ECCV 2024)
“Gaussian Grouping: Segment and Edit Anything in 3D Scenes.”
4. **FlashSplat** – Shen et al. (ECCV 2024)
“FlashSplat: 2D to 3D Gaussian Splatting Segmentation Solved Optimally.”
Trainingsfreie Segmentierung durch Optimierung auf 2D-Annotationen.
5. **Centerline-Extraktion** – Kerautret et al. (DGCI 2016)
“3D Geometric Analysis of Tubular Objects based on Surface Normal Accumulation.”
6. **COLMAP** – Schönberger, Frahm (CVPR 2016)
“Structure-from-Motion Revisited.”

Vergleichsstudien und Benchmarks

6. **GauU-Scene V2 Benchmark** – Inverse Korrelation PSNR vs. geometrische Genauigkeit
7. **ISPRS Annals 2025** – Geodätisch relevanteste Vergleichsstudie 3DGS vs. Photogrammetrie

8. **GeoRefGS** – “Towards Georeferenced 3DGS from UAV Platforms” (MDPI 2025)
Erwähnt als State-of-the-Art, nicht genutzt (kein öffentlicher Code)
9. **3DGS Applications Survey** – He et al. (2025)
“A Survey on 3D Gaussian Splatting Applications: Segmentation, Editing, and Generation.”
ArXiv: 2508.09977. Umfassendster Survey mit 50+ Segmentierungsmethoden.

Varianten (State-of-the-Art)

9. **Mip-Splatting** (CVPR 2024) – Anti-Aliasing für Multi-Scale
10. **2D Gaussian Splatting** (SIGGRAPH 2024) – Planare Gaussians
11. **GaussianPro** (ICML 2024) – Texturlose Regionen
12. **Scaffold-GS** (CVPR 2024) – Speicher-Reduktion
13. **DroneSplat** – Drohnenspezifische 3DGS-Pipeline

Large-Scale 3DGS (Skalierbarkeit)

14. **VastGaussian** – Lin et al. (CVPR 2024)
“VastGaussian: Vast 3D Gaussians for Large Scene Reconstruction.”
Automatische Partitionierung großer Szenen in Tiles mit seamless Merging.
15. **CityGaussian** – Liu et al. (2024)
“CityGaussian: Real-time High-quality Large-Scale Scene Rendering.”
Level-of-Detail für stadtweite 3DGS-Rekonstruktionen.

7. Installations-Checkliste

Voraussetzung: Hardware

- ✓ **NVIDIA RTX 4000 Ada – 20 GB VRAM** (vorhanden)
- ✓ **64 GB RAM** (vorhanden)
- ☐ NVIDIA Treiber $\geq 525.x$ (CUDA 11.3–11.8)
- ☐ Conda/Miniconda installiert
- ☐ Git installiert
- ☐ Visual Studio Build Tools (C++ Compiler)

COLMAP

- ☐ Pre-built Binary von <https://demuc.de/colmap/> (CUDA-Version)
- ☐ Entpacken und PATH setzen
- ☐ Test: `colmap -h`

gsplat (3DGS Training)

```
conda create -n gsplat python=3.10 -y
conda activate gsplat
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu118
pip install gsplat
```

SuGaR (Mesh-Extraktion)

```
git clone https://github.com/Anttwo/SuGaR.git --recursive
cd SuGaR
python install.py # Erstellt conda env "sugar"
conda activate sugar
```

- ☐ Windows-Pfade fixen: / → os.path.join() in relevanten Skripten

FlashSplat (Segmentierung – trainingsfrei)

```
git clone https://github.com/florinshen/FlashSplat.git
cd FlashSplat
pip install -r requirements.txt
```

Gaussian Grouping (Segmentierung – trainingsbasiert)

```
git clone https://github.com/lkeab/gaussian-grouping.git
cd gaussian-grouping
conda create -n gaussian_grouping python=3.8 -y
conda activate gaussian_grouping
conda install pytorch==1.12.1 torchvision==0.13.1 cudatoolkit=11.3 -c
pytorch
pip install plyfile==0.8.1 tqdm scipy wandb opencv-python scikit-learn lpips
pip install submodules/diff-gaussian-rasterization
pip install submodules/simple-knn
```

DGtal (Centerline)

```
git clone https://github.com/DGtal-team/DGtal.git
cd DGtal && mkdir build && cd build
cmake .. && make
```

Alternativ: Centerline-Algorithmus (Normalenakkumulation) in Python mit Open3D reimplementieren.

Georeferenzierung (eigenes Skript)

```
pip install numpy scipy # Kein extra Environment nötig
```

- ☐ CSV: ID, X_lokal, Y_lokal, Z_lokal, X_welt, Y_welt, Z_welt
- ☐ Helmert 7P mit scipy.spatial.transform
- ☐ Transformation auf alle Gaussian-Positionen anwenden

8. Evaluation-Framework (D1–D6)

Dim.	Metrik	Tool	PA	BA
D1: Geometrie	RMSE horiz./vert. [cm] vs. TLS	CloudCompare / numpy	–	✓
D2: Visuell	PSNR, SSIM, LPIPS	gsplat metrics	✓	✓
D3: Manuell	Visueller Vergleich	Screenshots	✓	✓
D4: ML-Erkennung	Precision, Recall, F1	sklearn	–	✓
D5: Kosten	Zeit, Kosten, Hardware	Tabelle	✓	✓
D6: Tile-Konsistenz	Overlap-RMSE [cm]	numpy	–	✓

Referenzdaten (verfügbar an der Hochschule)

RTK-GNSS: Für GCP-Einmessung und Georeferenzierung (Genauigkeit: < 1 cm).

Terrestrischer Laserscanner (TLS): Für dichte Referenz-Punktwolke des gesamten Testfelds. Daraus wird eine **TLS-Referenz-Centerline** extrahiert, gegen die die 3DGS-Centerline evaluiert wird (Linie-zu-Linie-Vergleich, nicht nur Punkt-zu-Punkt).

→ **Doppelte Referenz:** GCPs für Georeferenzierung, TLS für geometrische Evaluation.

9. Testfeld-Design (Campus, ca. 20–30 m)

Die Kamera wird auf ca. **2 m Höhe** geführt; bei einer realen Drohnenhöhe von 10 m ergibt sich ein **Skalierungsfaktor von 5×**. Die Testobjekt-Durchmesser erzeugen somit bei 2 m die gleiche Pixelabdeckung (GSD) wie reale Kabel bei 10 m Flughöhe:

Testobjekt	Ø Campus	Simuliert	Ø real (10 m Höhe)
Gartenschlauch (dick)	~40 mm	DC-Erdkabel	20 cm (200 mm)
Gartenschlauch (dünn)	~16 mm	Steuerkabel	8 cm (80 mm)
Paketschnur / Seil	~10 mm	Lichtwellenleiter	5 cm (50 mm)
Nylonschnur	~3 mm	Grenztest	< 2 cm
GCP-Targets	–	Passpunkte	8–12 Stück, RTK-GNSS (< 1 cm)
TLS-Scan	–	Referenz	Terrestr. Laserscanner (HS-Gerät)

10. Skalierbarkeit: Tile-basiertes Processing

Standard-3DGS ist für kleine bis mittlere Szenen optimiert ($\sim 50 \text{ m}^2$). Reale Kabeltrassen erstrecken sich über **100 m bis 1 km**. Die Lösung ist ein **georeferenziertes Tiling**, das in der Photogrammetrie Standard ist.

Tile-Strategie

1km Trasse --> Aufteilen in N Tiles a 50m (mit 10m Ueberlappung)

|
Pro Tile: COLMAP --> 3DGS --> Georef --> Segm. --> SuGaR --> Centerline

|
Alle Centerlines in georef. Koordinaten (UTM)

GIS-Merge: Spatial Join auf Ueberlappungszonen --> durchgehende Linie

Parameter	Empfehlung	Begründung	Auswirkung
Tile-Länge	50 m	Optimale 3DGS-Szenengröße	~500k Gaussians/Tile
Überlappung	10 m (20%)	Genug Kontext für COLMAP	Seamless Centerline-Join
GCPs pro Tile	3–4 Minimum	Für Helmert-Transformation	< 2 cm Georef.-Fehler

Merge-Strategie im GIS

1. Alle Tile-Centerlines als Polylines in ein GIS laden (gleiche Projektion)
2. In der Überlappungszone: **gewichtete Mittelung** der Positionen
3. **Spatial Join** auf < 0,5 m Abstand → durchgehende Linie
4. **Overlap-RMSE** berechnen: Abweichung der Centerlines beider Tiles in der Überlappung

Wissenschaftlicher Mehrwert: Der Overlap-RMSE ist eine **neue Evaluationsdimension** (D6), die die Konsistenz der Pipeline über Tile-Grenzen hinweg misst. Für das Campus-Testfeld (5–10 m) ist Tiling nicht notwendig – es kann aber als Proof-of-Concept getestet werden, indem man die Szene künstlich in 2–3 Tiles aufteilt.

Referenzen: VastGaussian (CVPR 2024) und CityGaussian (2024) zeigen, dass 3DGS auf Stadtmaßstab skaliert werden kann. Der hier gewählte manuelle Tiling-Ansatz ist für linienförmige Infrastruktur **einfacher und robuster**, da die Georeferenzierung die räumliche Konsistenz sicherstellt.

11. Risikomatrix

Risiko	Wahrsch.	Impact	Mitigation
SuGaR Windows-Pfade	Hoch	Niedrig	Trivial fixbar mit pathlib
Kabel nicht erkannt	Mittel	Hoch	Fallback: manuelle Farb-Filterung
Centerline schlecht	Mittel	Hoch	Alternative: Open3D Skeleton
VRAM < 12 GB	<i>Entfällt</i>	–	RTX 4000 Ada mit 20 GB vorhanden
RTK-GNSS / TLS nicht verf.	<i>Gering</i>	Mittel	Geräte an HS vorhanden, frühzeitig reservieren
Mesh verrauscht	Mittel	Mittel	SuGaR <code>dn_consistency</code> nutzen
Szene zu groß	Mittel	Mittel	Tile-basiertes Processing (50 m Tiles)

Externe Machbarkeitsanalyse (ValyuAI)

Eine unabhängige Machbarkeitsprüfung bestätigt für dieses Forschungsdesign die Statusfreigabe **START GRÜN**.

- **Machbarkeit:** Sehr hoch (alle Komponenten verifiziert, Hardware mit RTX 4000 20 GB ausreichend).
- **Wissenschaftlicher Mehrwert:** Sehr hoch. Es schließt eine echte Forschungslücke (3DGS-Genauigkeit für Centerline-Extraktion).

- **Fallbacks (Risiko-Minimierung):** Falls *Gaussian Grouping* zu rechenintensiv wird, kann auf *FlashSplat* reduziert werden. Falls *DGtal* hängt, kann das Mesh exportiert und extern (z.B. Open3D) verarbeitet werden.
- **Publikationspotenzial:** Es sind bis zu 3 Papers ableitbar (Segmentierungs-Vergleich, Scan-to-BIM Genauigkeitsanalyse gegen TLS, Open-Source Toolkit Report).

12. Diskussionspunkte für den Dozenten

1. **Scope PA vs. BA:** PA = Datenerhebung + Vergleich (MVS vs. 3DGS). BA = Scan-to-BIM Pipeline.
2. **Segmentierung:** Dreistufiger Vergleich: Farb-Filterung vs. FlashSplat vs. Gaussian Grouping.
3. **Georeferenzierung:** Klassische Helmert-Transformation über COLMAP GCPs.
4. **Skalierbarkeit:** Tile-basiertes Processing (50 m Tiles, 10 m Overlap) für Trassen bis 1 km. Machbarkeitstest auf Campus durch künstliches Aufteilen in 2–3 Tiles.
5. **Referenzdaten:** RTK-GNSS (GCPs) + TLS-Scan (Referenz-Centerline) – beides an der HS verfügbar.
6. **Testfeld:** Campus ausreichend, oder Drohnenflug?
7. **KI-Nutzung:** Dokumentation gemäß PABA-Hinweise Kap. 3.5.
8. **Hardware:** RTX 4000 Ada (20 GB) + 64 GB RAM vorhanden.
9. **Centerline:** DGtal (C++) oder eigene Python-Reimplementierung?
10. **Survey:** He et al. (2025) als Grundlage für SotA-Kapitel (50+ Methoden).